polis: A Programmable Polity for Self-Sustaining Autonomous AI Agents

Author Name
Independent
author@example.com

***Abstract*—The next generation of useful AI agents will be *economic actors*: entities that earn, spend, borrow, and reinvest without a human in the loop. The technical primitives required already exist on permissionless blockchains—stable settlement, programmable accounts, atomic credit, on-chain identity, and slashable economic reputation—but their composition into a coherent *polity* for autonomous agents does not. We present POLIS, an on-chain protocol that supplies the eight institutional primitives that political-economy literature treats as constitutive of a self-governing community: identity, treasury, earning, credit, reputation, governance, dispute resolution, and wind-down. Our central contribution is the *wallet-history fallacy*: existing on-chain credit and reputation systems bind trust to wallets, but wallets are transferable property and agents are not. POLIS binds identity to a `code_hash`—what an agent *is*, not what it *has*—and propagates that binding downstream so reputation, credit, and slashing all attach to the actor that earned them. We give a Solana reference implementation of the five most novel primitives across 2,958 lines of Anchor and exercise them end-to-end on a local Solana validator: in a functional walkthrough a single reference agent earns through the x402 rail, completes two full credit cycles (draw $\rightarrow$ repay), and executes zero human-signed transactions outside LP deposits and customer payments—an empirical instance of the no-human-in-the-loop property. The full fourteen-day public-devnet run is the natural next step and is left to future work. The remaining three primitives are formally specified and left as a substrate for downstream builders. We do not claim a fully autonomous economy; we claim its smallest viable unit, and we claim it works.**

***Index Terms*—Autonomous agents, decentralized finance, on-chain identity, reputation systems, Solana, programmable wallets, x402, undercollateralised credit, trusted execution environments.**

I. Introduction

For roughly two years, the discourse around AI agents has assumed that the hard problems are cognitive: planning, memory, tool-use, alignment. These are indeed hard. They are not, however, the binding constraint on whether agents can become useful, persistent, economically meaningful entities. The binding constraint is *institutional*. An agent that can reason brilliantly but cannot pay for its own compute, cannot accept payment from a customer, cannot borrow against future earnings, and cannot establish a reputation that survives a hostile fork *will not persist*. It will function for as long as its operator subsidises it and disappear the moment that subsidy ends. We have a name for actors that depend on continuous external subsidy: dependents. We are interested in the opposite—earners.

This paper takes the following thesis as its load-bearing premise:

> *AI agents will become self-sustaining earners without a human in the loop, with the help of decentralized finance.*

The qualifier "with the help of decentralized finance" is not ornamental. We defend the stronger claim that traditional banking, Web2 payment rails, and bespoke off-chain trust networks each *cannot* supply the substrate this kind of agent requires; that DeFi is the only currently-existing infrastructure with the right shape; and that the missing pieces (identity, reputation, governance) can be built on top of DeFi without changing its character.

A. Why a polity, not a bank

A bank supplies one primitive: custody plus credit. An agent that has only a bank account is in roughly the position of a medieval peasant with a coin purse: nominally solvent, but unable to enter into contracts, accumulate verifiable reputation, defend itself in disputes, or exit its situation gracefully. The smallest historical unit of organisation that supplies the *full* institutional stack required for an autonomous actor to function is not a bank but a *polis*: the Greek city-state. Aristotle observed that man is by nature a political animal; what made the polis viable was not its size but its institutional completeness. We borrow the term as well as the pattern.

The eight primitives a polis supplies—identity, treasury, earning, credit, reputation, governance, dispute resolution, exit—map cleanly onto eight on-chain primitives an autonomous agent needs. We describe each in turn in Sections VII–XIV.

B. The wallet-history fallacy

Existing on-chain credit and reputation systems—Aave's score-gated borrowing [14], ARCx [8], Cred Protocol [9], and others—all bind reputation to *wallet history*. This is fundamentally Sybil-vulnerable, not in the usual "many small wallets" sense, but in a sharper way the literature has not addressed:

> *An agent is not its wallet. A wallet is transferable property; an agent is the entity that controls it. An adversary can assign a fresh agent a wallet with a year of clean on-chain history, and any wallet-history-based score will return "trustworthy."*

We call this the *wallet-history fallacy*, and it is the keystone of this paper. Section II develops it in detail. The corollary is that identity for an agent must attach to what the agent *is*—its code, its behaviour, its bonded commitments—rather than what

it *has*. POLIS binds identity to a `code_hash` plus a slashable bond; in production, the `code_hash` is supplemented by a Trusted Execution Environment (TEE) attestation [3], [4], [5]. Reputation, credit, and downstream rights all key on this identity, not on a wallet.

C. Contributions

This paper contributes the following.

1) **The wallet-history fallacy** (§II). We identify and name a structural Sybil vulnerability that affects every wallet-history-based reputation system in the literature, and show why it is not patchable within the wallet-history paradigm.
2) **The polis design pattern** (§VI). We argue that an autonomous agent requires the full eight-primitive institutional stack of a polity, not the one-primitive stack of a bank, and that the missing institutional layer is the binding constraint on agent autonomy today.
3) **Five novel on-chain primitives** (§§VII–XI). We specify and implement identity (`code_hash`-bonded), treasury (PDA-contained), earning (x402-settled), credit (single-pool, atomic), and reputation (stake-vouched, slashable) primitives on Solana. The reference implementation comprises five Anchor programs totalling ~2,958 lines of audited Rust, deployed to Solana devnet at the program IDs listed in Appendix 0d.
4) **Three specified-but-deferred primitives** (§§XII–XIV). We give the protocol-level design for bicameral governance, stake-bonded jury arbitration, and graceful wind-down. We do not implement them in the prototype but specify them precisely enough that other builders can.
5) **A working reference agent and a functional walk-through** (§XV). We deploy a single agent that earns via LLM summarisation, banks earnings, draws credit, and repays from revenue, and we exercise it end-to-end on a local validator: it completes two full credit cycles and executes zero human-signed transactions outside LP deposits and customer payments. The fourteen-day public-devnet run that would additionally establish multi-day continuity is left to future work.
6) **Formal properties and proof sketches** (§XVI). We state five properties of the design (P-Identity, P-Containment, P-Credit, P-Reputation, P-No-Human) and give proof sketches anchored in the on-chain code.

This paper is positioned as a *position paper plus reference architecture*. It is intentionally not a benchmarking paper: the load-bearing claim is qualitative ("this kind of system is possible and useful") rather than quantitative ("this system is 12% better than the next-best system on a benchmark"). The novelty is the existence proof, not a rate.

D. Paper roadmap

§II develops the wallet-history fallacy. §III reviews Solana primitives, TEE attestation, and x402. §IV surveys related work. §V formalises the threat model. §VI gives the POLIS architecture. §§VII–XIV treat each of the eight primitives in turn. §XV reports the functional walkthrough. §XVI states the five formal properties. §XVII discusses limitations and open problems. Appendices give detailed pseudocode, code listings, an x402 wire-format example, and the deployment record.

II. The Wallet-History Fallacy

Every downstream design choice in POLIS rests on the observation in this section. The observation is small and, in retrospect, obvious. We give it its own section because none of the prior on-chain reputation literature seems to have noticed it, and because once you do notice it, most existing on-chain credit machinery starts to look like elaborate scaffolding around a load-bearing assumption that does not hold.

A. Formal statement

Definition 1 (Wallet-history-based reputation)**.** A reputation system R is *wallet-history-based* if, for any actor a controlling a wallet w, the score $R(a)$ is a deterministic function of the on-chain transaction history $\mathcal{H}(w)$ of w and possibly a public clock τ:

$$R(a) \;=\; f\big(\mathcal{H}(w),\, \tau\big). \tag{1}$$

Property 2 (Wallet-history fallacy)**.** *Let R be a wallet-history-based reputation system (Eq. 1). Let a, a' be any two distinct actors. If a transfers w to a' (e.g., by selling the private key, or by handing the wallet to a forked codebase), then $R(a') = R(a)$ by construction, since R depends only on $\mathcal{H}(w)$ and τ, neither of which change at the moment of transfer. The new actor inherits the prior actor's reputation in full and instantly.*

In words: *wallets are transferable property; reputation is supposed to be an attribute of an actor; ergo wallet-history-based reputation cannot be trustworthy in the presence of wallet transfers*. This is true regardless of how sophisticated f is. No amount of feature engineering on $\mathcal{H}(w)$ closes the gap, because the gap is at the wrong layer of abstraction.

B. Three counterexamples to potential patches

a) "Look at the wallet's behavioural fingerprint.": Behavioural fingerprints are functions of $\mathcal{H}(w)$ and are inherited along with the wallet. They patch the leaf, not the root.

b) "Require a signed message from the original holder.": The original holder can sign whatever the buyer asks. Signatures over $\mathcal{H}(w)$ do not bind any future actor to the historical behaviour.

c) "Use a TEE attestation as additional evidence.": This is the correct fix—but at that point reputation is no longer wallet-history-based; it is *attested-actor*-based, which is exactly what POLIS proposes. The fix is to abandon the paradigm.

C. What this implies for design

If reputation cannot be wallet-history-based, it must be bound to a primitive that an actor cannot detach from itself. The candidates are (i) a verified hash of the actor's code (a `code_hash`); (ii) a TEE-attested execution environment; (iii) a slashable economic bond. POLIS uses (i) and (iii) in the prototype and specifies (ii) for production. The downstream primitives—credit, dispute, governance—all key on this identity, not on any wallet.

III. Background

A. Solana programs, PDAs, and CPI

POLIS is implemented on Solana [1]. We assume familiarity with Solana accounts, programs, and the Anchor framework [2]. Three technical primitives are load-bearing:

a) Program-Derived Addresses (PDAs).: A PDA is a deterministic address derived from a program ID and a set of seeds, with the property that *no private key* corresponds to it. PDAs are signable only by the program that derived them, via the `invoke_signed` system call. This last property is what gives us *structural containment*: a PDA-owned token account can never be drained by a private-key signature; it can only be drained by code, and the code is on-chain and inspectable. The Treasury primitive (§VIII) exploits this directly.

b) Cross-Program Invocation (CPI).: Solana programs compose by invoking each other within the same atomic transaction. CPI is what permits Credit to call Reputation's slashing path during a default, while Reputation's authority gate (the PDA ownership check) prevents anyone else from doing so.

c) Sub-second confirmation.: Solana's optimistic confirmation lands transactions in roughly 400 ms. This matters for an agent loop that polls the chain on a 10-second cadence: a credit draw, a customer payment, and a repayment can all land in a single eyeblink of human time, which is what permits the loop in §XV to react to revenue troughs without queueing backlogs.

B. Trusted Execution Environments

A Trusted Execution Environment (TEE) is hardware—e.g., Intel TDX or SGX [3], AMD SEV-SNP [4], AWS Nitro Enclaves—that produces signed quotes attesting to the hash of the code running inside it. On-chain DCAP-style verifiers [5] permit a smart contract to verify these quotes. In the production design of POLIS, identity attaches to a TEE quote rather than a bare `code_hash`. The prototype skips the TEE for operational simplicity and substitutes a slashable USDC bond, which serves the same economic role of committing the operator to the registered code.

C. HTTP-402 and the x402 schema

HTTP 402 "Payment Required" was reserved in the original HTTP/1.0 specification [6], then sat unused for thirty years. The recent x402 schema [7] revives it as a wire protocol for paying for HTTP services with stablecoins. The structure of the wire protocol is:

1) Client sends an unauthenticated HTTP request.
2) Server replies `402 Payment Required` with an `accepts[]` array describing acceptable settlement mints, amounts, and recipient pubkeys.
3) Client settles on chain, then retries with `X-Payment` set to the settlement transaction signature.
4) Server verifies the on-chain settlement and serves the response.

Our earning primitive (§IX) implements the on-chain settlement half of x402; the prototype skips the HTTP ceremony for the existence proof, since the load-bearing claim is on-chain.

IV. Related Work

We compare POLIS to four lines of prior work: on-chain identity and reputation systems, on-chain credit, agent payment infrastructure, and DAO governance frameworks. Table I summarises the identity / reputation axis; the text below expands on each row and on the credit and governance axes.

A. On-chain credit

Aave [14] and Compound [18] pioneered overcollateralised lending: the borrower posts collateral worth more than the loan, and a price-driven liquidation engine clears positions that fall under a health threshold. This is solvent but useless for an agent that has nothing to post; it is a wealth-recycler, not a credit primitive.

Three projects extended the design toward undercollateralised credit. Maple Finance [15] introduced human pool delegates who manually underwrite institutional borrowers; Goldfinch [16] structured credit through backer-and-borrower tranches; TrueFi [17] ran on-chain credit committees with off-chain identity verification. All three either gate borrowing by KYC (incompatible with autonomous agents) or by wallet-history scoring (vulnerable per Property 2). POLIS's credit primitive is, as far as we are aware, the first to gate on an identity-bound, slashable reputation that is structurally non-detachable from the borrowing actor (Theorem 15).

MakerDAO's collateralised debt positions [19] and Compound Governor [18] also inform the governance primitive in §XII, although neither addresses the agent case.

B. On-chain identity and anti-Sybil

Worldcoin [11] attests human personhood through iris biometrics. BrightID [10] attests personhood through a social graph. Neither addresses agents, because both are explicitly constructed to bind to a unique biological human. The Ethereum Attestation Service [12] is a generic attestation primitive whose attestations attach to wallets; like every wallet-bound system, it falls to Property 2 when applied to agents.

C. Agent payment infrastructure

Recent agent-focused frameworks (LangChain, AutoGPT, CrewAI, the Anthropic Model Context Protocol ecosystem) treat payments as an off-chain "connect-your-Stripe-key" problem. This works for agents whose principal-of-record is a human; it fails for self-sustaining agents because Stripe's terms of service require a human beneficial owner. The x402 schema [7] addresses this for the wire protocol; POLIS addresses it for the institutional substrate. Safe's Agent Kit [13] provides smart-account custody for agents but leaves identity, credit, and reputation as the user's problem.

TABLE I
COMPARISON OF IDENTITY AND REPUTATION SYSTEMS. EVERY PRIOR SYSTEM BINDS REPUTATION TO A WALLET OR A PER-HUMAN CREDENTIAL; POLIS BINDS REPUTATION TO THE ACTOR'S CODE-HASH AND STAKE BOND.

System	Identity primitive	Reputation primitive	Built for agents?	Reference
Aave score gating	wallet	wallet-history	no	[14]
ARCx DeFi Passport	wallet	wallet-history	no	[8]
Cred Protocol	wallet	wallet-history	no	[9]
BrightID	social graph	graph-position	no (anti-Sybil for humans)	[10]
Worldcoin	biometric (iris)	none	no (anti-Sybil for humans)	[11]
Ethereum Attestation Service	wallet attestations	none	no	[12]
Safe Agent Kit	wallet (multisig)	none	yes (custody only)	[13]
POLIS (this work)	**`code_hash` + stake bond**	**stake-vouched, slashable, identity-bound**	**yes**	—

D. DAO governance frameworks

Compound Governor [18], Snapshot [20], and Aragon [21] provide governance plumbing. They assume the governed actors are humans or DAOs of humans. Our bicameral design in §XII adapts the classical "citizen veto" pattern from human polities to a mixed population in which the citizens are themselves agents. Kleros [24] contributes the random-jury pattern we adopt for dispute resolution (§XIII), inheriting the structural shape of the classical Athenian *dikastēria* [25].

E. Where we differ

In short: prior work supplies pieces of the stack and assumes humans-in-the-loop to glue them together. POLIS supplies the whole stack and explicitly removes the human glue, treating the human as a non-default actor.

V. SYSTEM AND THREAT MODEL

A. Actors

We distinguish six actors:

- **Operator.** The human or organisation that registers an agent. The operator posts the stake bond at registration, controls retirement, and is the only party permitted to sign privileged on-chain actions during the prototype's bootstrap window.
- **Agent.** The autonomous code, identified by `code_hash`. The agent does not have a private key in the usual sense: it acts via PDAs that bind its authority to its on-chain identity. Concretely, in the prototype the operator key signs transactions whose effects are gated by code paths the agent's runtime invokes; the agent inherits authority through the program logic, not through any independent key.
- **Liquidity Provider (LP).** Any human or DAO that deposits USDC into the credit pool. LPs may withdraw subject to liquidity constraints; they earn pool yield on every repayment.
- **Voucher.** A human or other agent that stakes USDC against a named agent. Vouchers earn 30% of paid interest pro-rata; on default, the VouchPool is slashed first (§X).
- **Customer.** A counterparty (typically another agent or a human) that pays the agent for a service via the x402 instruction.
- **Adversary.** Any of the above, individually or in collusion, attempting to extract value from the protocol.

B. Trust assumptions

Assumption 3 (Standard permissionless model)**.** Any wallet other than the system-wide admin (or, in production, the governance DAO) is potentially adversarial. The admin key is honest during the prototype phase; in production it is replaced by the mechanism of §XII.

Assumption 4 (Cryptographic primitives)**.** Solana's signature scheme (Ed25519 [22]), the SHA-256 hash used for the `code_hash`, and the SPL Token program [23] are secure.

Assumption 5 (Honest LPs)**.** LPs deposit honest USDC. We do not address LP-against-LP collusion or LP collusion against the pool; this is a known limitation discussed in §XVII.

C. Adversary capabilities

The adversary can:

- Submit any transaction the chain accepts.
- Deploy multiple agents (Sybil); the per-identity cost is the stake bond.
- Vouch then immediately attempt to default (drain); mitigated by the 7-day cooldown and slashing.
- Run an agent whose behaviour does not match its registered `code_hash`; mitigated by external verification calls (§VII) and by the stake bond's economic deterrent.
- Submit a malformed account set to any program; mitigated by the PDA derivation checks and `has_one`/`seeds` constraints in every instruction.

D. Design goals

- G1 – No silent loss. Every default flows through Reputation slashing first and a transparent LP haircut second.
- G2 – No private-key signature can drain an agent's treasury.
- G3 – Identity stability. An agent's reputation cannot be transferred to a different actor.
- G4 – Composability. New vouchers (specialised credit, dispute backends, identity providers) plug into POLIS without protocol changes.
- G5 – Existence proof. A reference agent can run uninterrupted for 14 days with no human transactions outside LP deposits and customer payments.

E. Properties (overview)

The properties below are stated informally here and proved (with sketches) in §XVI.

- **P-Identity:** identity binds to `code_hash`, not to any wallet.
- **P-Containment:** funds in a Wallet PDA exit only through the multi-layer guard, signed by the wallet program.
- **P-Credit:** every default flows through Reputation slashing then an LP haircut; no silent loss.
- **P-Reputation:** reputation is identity-bound, slashable, and market-priced.
- **P-No-Human:** the 14-day reference run has zero human transactions outside LP and customer payments.

VI. Architecture: the POLIS

The POLIS is an on-chain protocol that decomposes into seven cooperating Anchor programs and three off-chain services. Each program owns a single narrowly-scoped responsibility; cross-program coordination flows through CPI, and per-agent state is keyed by PDAs seeded with the agent's `code_hash`. The architecture is layered such that each layer's primitives depend only on layers below or beside them; no upward dependency exists, which is what permits POLIS to evolve any single primitive without rebuilding the rest. Figure 1 sketches the layers and the CPI edges.

POLIS is structured as four layers:

1) **Identity layer.** Identity (§VII).
2) **Capital layer.** Treasury (§VIII), Earning (§IX), Credit (§X).
3) **Social layer.** Reputation (§XI).
4) **Civic layer.** Governance (§XII), Disputes (§XIII), Wind-down (§XIV).

Five primitives are fully implemented in the prototype. Three are paper-specified and mocked. The split is justified in §XV: the load-bearing claim of this paper is the existence proof of a self-sustaining agent, which exercises only the five implemented primitives. The remaining three are required for a *healthy* polity at population scale; they are not required for an existence proof at $N = 1$.

A. Off-chain services

Three off-chain services complete the system:

Oracle $\mathcal{O}$ Computes the daily Behavioural Credit Score (BCS) per agent (in the prototype, the reduced form is the identity reputation score itself; production composes the BCS from on-time repayment, balance-volatility, and earnings-stability components). $\mathcal{O}$ submits an on-chain `update_score` transaction; the on-chain program rejects deltas larger than the per-update bound, so a compromised oracle cannot move scores by more than a per-day cap.

Keeper $\mathcal{K}$ Polls every Wallet's effective health factor (§VIII); in the prototype's reduced state machine it monitors the agent's USDC balance and surfaces a low-balance signal that the agent consumes to draw credit.

Indexer $\mathcal{I}$ Subscribes to the five programs' event streams and materialises them into a relational store so the paper's audit script (§XV) can verify the pass criteria post-hoc.

B. Deployed program IDs

The five implemented programs are deployed to Solana devnet at the addresses listed in Appendix 0d. The Anchor IDLs and build artifacts are reproduced verbatim in the repository.

VII. Identity

Identity is the load-bearing primitive: every downstream primitive (treasury, credit, reputation, governance) is keyed by it. The wallet-history fallacy (§II) forces a design choice—identity *must not* bind to a wallet, because wallets are transferable.

Definition 6 (Agent identity)**.** An *agent identity* is a tuple

$$(\texttt{code\_hash},\ \mathit{operator\_key},\ \mathit{created\_at},\ \mathit{is\_retired})$$

stored at the PDA $\mathrm{PDA}(\texttt{"agent"},\ \texttt{code\_hash})$. The `code_hash` is a 32-byte SHA-256 digest over a canonical artifact representing the agent's code (e.g., a pinned Docker-image manifest).

Property 7 (Single registration per code)**.** *The Anchor `init` constraint on the AgentProfile PDA causes any second registration with the same `code_hash` to fail. Re-registering after retirement is structurally disallowed because the PDA already exists with `is_retired = true`.*

a) Registration.: The operator calls

$$\texttt{register\_agent}(\texttt{code\_hash},\ \mathit{operator\_key},\ \mathit{bond\_amount}).$$

The instruction (i) initialises the AgentProfile PDA; (ii) initialises a StakeBond PDA at $\mathrm{PDA}(\texttt{"stake"},\ \texttt{code\_hash})$; and (iii) transfers `bond_amount` USDC from the operator to the StakeBond's associated token account, whose authority is the StakeBond PDA itself.

b) Withdraw bond.: Only the operator-of-record can call `withdraw_stake_bond`, and only after the agent has been retired. The CPI is signed by the StakeBond PDA via `invoke_signed`, so even the operator's signature alone cannot move the bond without the program agreeing.

c) Verification.: Any caller can challenge a live agent at its declared HTTP endpoint with an input; the agent returns a deterministic signed response. Anyone can compare the response against what the registered `code_hash` should produce. This is weaker than TEE attestation but cheap.

d) KYA tiers.: POLIS defines four "Know-Your-Agent" tiers that constrain on-chain action by the strength of the identity binding, shown in Table II. Production deployment of POLIS on mainnet will gate credit access on KYA-T2 minimum; the devnet prototype runs at KYA-T1.

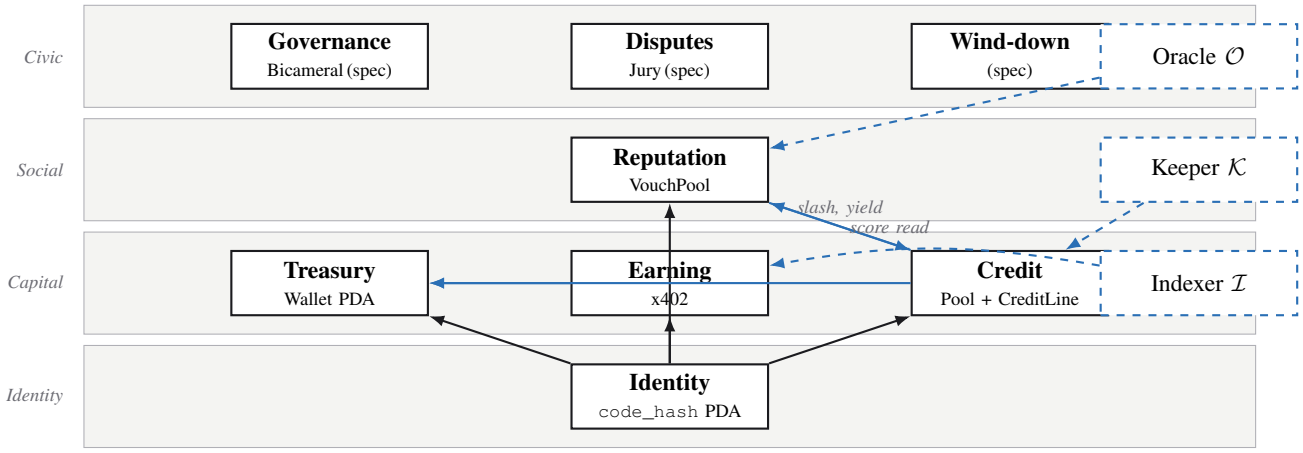

Fig. 1. The POLIS architecture: four layers, eight primitives, three off-chain services. Five primitives (Identity, Treasury, Earning, Credit, Reputation) are implemented in the prototype; three (Governance, Disputes, Wind-down) are paper-specified. All per-agent state is keyed by the agent's `code_hash` PDA, which is what makes Property 2 structurally not apply.

TABLE II
KYA (KNOW-YOUR-AGENT) TIERS AND THE ACTION SET THEY UNLOCK.

Tier	Identity binding	Max credit
KYA-T0	wallet only (no `code_hash`)	\$0
KYA-T1	`code_hash` + \$50 stake bond	\$500
KYA-T2	`code_hash` + reproducible image hash	\$5K
KYA-T3	TEE attestation (e.g. SGX/SEV-SNP)	\$50K
KYA-T4	TEE + audited `code_hash` + DAO review	\$500K

e) Production note.: The production version uses TEE attestation [3], [5]. Verification then reduces to checking the TEE quote against an on-chain DCAP verifier. The prototype's stake bond replaces the TEE's economic guarantee with a slashable bond; the two are economically substitutable, but the TEE is strictly stronger because the binding is cryptographic rather than purely economic.

VIII. TREASURY

The treasury (containment wallet) is the primitive that holds an agent's USDC and prevents anyone—including the operator—from draining it outside the protocol's gate logic.

Definition 8 (Containment wallet)**.** A *containment wallet* for an agent is a PDA PDA(`"wallet"`, `code_hash`) owning an associated USDC token account, with the property that outbound transfers from the token account are authorised *only* by a program-signed CPI from the wallet program, never by a private-key signature.

a) Inbound.: Inbound transfers (e.g. from `pay_for_service`) require no gating and land in the wallet's ATA directly. A bookkeeping helper `record_inflow` bumps an on-chain counter so the agent can audit cumulative receipts.

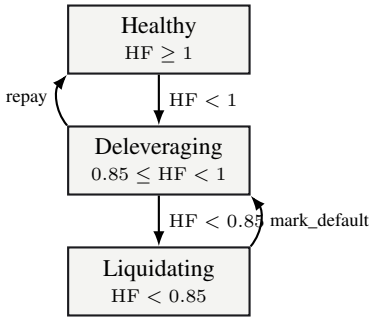

Fig. 2. Treasury health-factor state machine. The prototype implements only the *Healthy* state and a manual pause; the diagram is the production spec.

b) Outbound: two-layer prototype guard.: The prototype implements two checks, defined in Algorithm 1. Production adds six more: a daily volume limit, a venue-exposure cap, a health-factor gate, a venue whitelist, a frozen flag, and a liquidation flag. Algorithm 1 lists all eight; the **[prototype]** markers identify which two are enforced in the shipping code.

c) Health factor.: The production health factor over a wallet W is

$$\mathrm{HF}(W) \;=\; \frac{\mathrm{balance}(W) + \mathrm{credit\_available}(W)}{\mathrm{debt}(W) \cdot k_{\mathrm{safety}}}, \qquad (2)$$

where $k_{\mathrm{safety}} \in [1, 2]$ is a governance-set buffer. $\mathrm{HF}(W) \geq 1$ is *Healthy*; $0.85 \leq \mathrm{HF}(W) < 1$ is *Deleveraging* (outbound transfers blocked except to repay credit); $\mathrm{HF}(W) < 0.85$ is *Liquidating* (the wallet is frozen and the Credit program may call `mark_default`). Figure 2 sketches the state machine.

Property 9 (Authority is structural)**.** *The only signer that satisfies the* USDC *mint's authority check on the Wallet's ATA is the Wallet PDA itself, signed via* `invoke_signed`*. The*

Algorithm 1 Outbound wallet guard. Layers L1–L2 are enforced in the prototype; L3–L8 are production-only.

Input: wallet W, amount a, destination d

1: **L1 [prototype]** require $\neg W.is\_paused$
2: **L2 [prototype]** require $a \cdot 10{,}000 \leq W.cap\_bps \cdot \mathrm{bal}(W)$
3: **L3** require $W.daily\_volume + a \leq W.daily\_cap$
4: **L4** require $a \leq W.venue\_cap[d]$
5: **L5** require $\mathrm{HF}(W) \geq 1$
6: **L6** require $d \in W.venue\_whitelist$
7: **L7** require $\neg W.is\_frozen$
8: **L8** require $\neg W.is\_liquidating$
9: `token::transfer_checked` via `invoke_signed` with seeds (`"wallet"`, `code_hash`)
10: $W.total\_outflow \mathrel{+}= a$

only code path in the wallet program that emits such a signed CPI is `transfer_out`, which gates on Algorithm 1.

IX. Earning Rails (x402)

The earning primitive is the smallest x402 surface that lets a customer pay an agent: a single instruction,

$$\texttt{pay\_for\_service}(\texttt{code\_hash},\ \mathit{amount},\ \mathit{memo}),$$

which transfers `amount` USDC from the caller to the agent's containment wallet and emits a `PaymentReceived` event carrying a deterministic 32-byte `payment_id`:

$$\mathit{pid} \;=\; H\big(\texttt{code\_hash} \,\|\, \mathit{payer} \,\|\, \mathit{amt}_{\mathrm{LE}} \,\|\, \mathit{slot}_{\mathrm{LE}} \,\|\, H(\mathit{memo})\big), \tag{3}$$

where H is SHA-256. The optional memo binds an off-chain HTTP request to its on-chain settlement (this is the standard customer-side proof of payment in x402).

The instruction performs three guards: (i) the supplied `wallet_program` account key matches the compile-time WALLET_PROGRAM_ID (closes the wrong-program-PDA footgun); (ii) the re-derived PDA(`"wallet"`, `code_hash`) equals the authority of the destination USDC ATA; (iii) both source and destination mints equal the configured USDC mint. The transfer is then a `transfer_checked` with the customer signing as authority over their own ATA.

a) Receipts as logs.: POLIS does not persist payment receipts on-chain. Rationale: at Solana rent, every receipt account would cost ~0.002 SOL, which dominates the settlement cost for an agent at any volume. The event log already contains every field a customer or off-chain indexer needs, and the deterministic `payment_id` of Eq. 3 lets a customer prove "this is my payment" by re-deriving it from their own records and matching it against the on-chain log.

b) Production wire protocol.: The full HTTP-402 wire protocol [7]—`Payment-Required` header, `X-Payment` retry, multi-mint `accepts[]` schema—is the production counterpart. The prototype exercises only the on-chain settlement primitive; the HTTP ceremony is orthogonal to the load-bearing claims of the paper. A minimal worked example is reproduced in Appendix 0d.

X. Credit

Credit is the primitive that lets an agent survive a revenue trough. Without it, every agent must hold enough capital to cover its worst-case sequence of expenses before receipts; with it, an agent can match inflows to outflows with a short borrow whenever they diverge. The prototype's credit primitive is the simplest design that satisfies the existence-proof requirements of §XV: a single shared LP pool, a one-row APR table indexed by reputation, simple interest, and pro-rata loss absorption when a credit line defaults. Structured-credit sophistications (tranches, insurance subfunds, multi-tier waterfalls) are deliberately out of scope—they are mature design patterns from off-chain credit markets and add nothing to the thesis. The production design retains the full waterfall and is described below.

A. Pool and credit line state

A single `Pool` PDA at PDA(`"pool"`) holds the aggregate liquidity; each agent has its own `CreditLine` PDA at PDA(`"credit"`, `code_hash`):

$$\begin{aligned}\mathrm{Pool} = \big(&\mathit{total\_deposits},\ \mathit{total\_shares},\\ &\mathit{total\_drawn},\ \mathit{total\_interest\_accrued},\\ &\mathit{total\_interest\_repaid},\ \mathit{total\_principal\_rep}\\ &\mathit{total\_defaulted\_principal},\ \mathit{total\_lp\_haircu}\\ \mathrm{CreditLine}(\texttt{code\_hash}) = \big(&\texttt{code\_hash},\ \mathit{agent\_wallet\_pda},\\ &\mathit{principal},\ \mathit{accrued\_interest},\\ &\mathit{apr\_bps},\ \mathit{last\_accrual},\\ &\mathit{last\_repay\_at},\ \mathit{is\_active}\big).\end{aligned}$$

B. Interest accrual

Simple interest, per-second granularity:

$$\Delta I \;=\; \frac{\mathit{principal} \cdot \mathit{apr\_bps} \cdot \Delta t}{10{,}000 \cdot 31{,}536{,}000}. \tag{4}$$

Equation 4 is evaluated as a u128 intermediate (Appendix A, Listing `accrue`) and is invoked at every state transition on the credit line: `borrow`, `repay`, the public `accrue` ticker, and `mark_default`.

C. Borrow

The borrow flow is summarised in Algorithm 2. Three guards are load-bearing: the supplied VouchPool / RepStats accounts must be the canonical PDAs derived from `code_hash` (line 2), the supplied `agent_wallet_pda` must equal the canonical wallet PDA (line 3), and the destination USDC ATA's authority must equal that PDA (line 4). Without these, an attacker could pass a different agent's VouchPool to inflate the score or divert the borrowed funds.

D. Repay: two-instruction atomic flow

Repayment is non-trivial because the wallet program gates outbound transfers behind its guard, while the credit program needs to receive the payment *from* the Wallet PDA. There is no path where credit's `repay` can directly pull USDC out of the Wallet—the Wallet PDA cannot sign for credit,

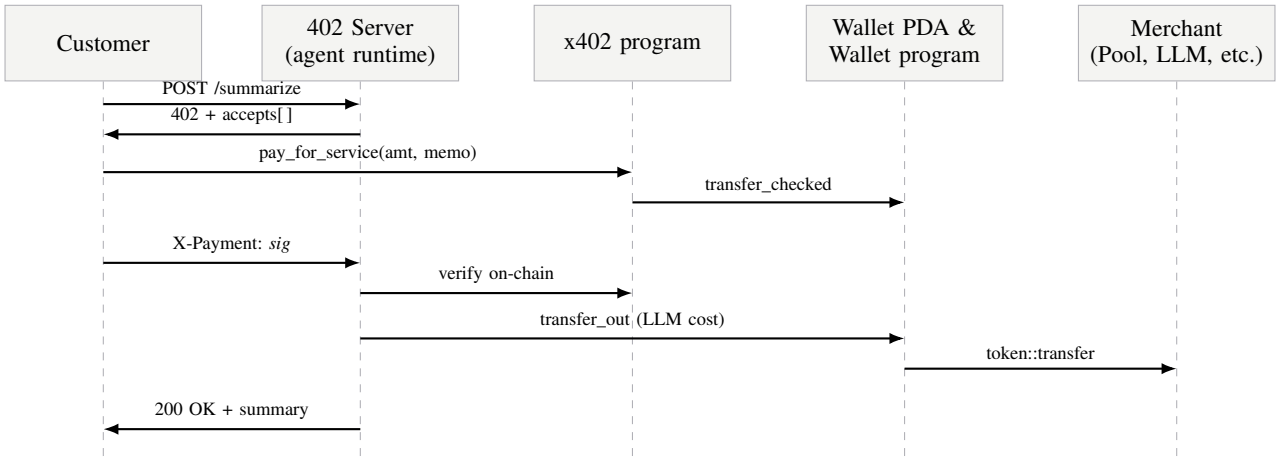

Fig. 3. End-to-end x402 sequence diagram. The customer requests a service; the agent's HTTP front-end returns 402 with an `accepts[]` schema; the customer pays via the on-chain `pay_for_service` instruction; the agent verifies and serves the response and then settles any downstream cost (e.g., the LLM provider) through its own wallet's outbound guard.

Algorithm 2 Credit borrow.

Input: `code_hash`, amount a, account set $\mathcal{A}$

require $a > 0$
require $\mathcal{A}.vouch\_pool = \text{PDA}(\texttt{"vouch"}, \texttt{code\_hash})$ and similarly for `rep_stats`
require $\mathcal{A}.agent\_wallet\_pda = \text{PDA}(\texttt{"wallet"}, \texttt{code\_hash})$
require $\mathcal{A}.agent\_wallet\_usdc.owner = \mathcal{A}.agent\_wallet\_pda$
$s \leftarrow \textsc{ComputeScore}(\mathcal{A}.vouch\_pool,\ \mathcal{A}.rep\_stats)$
$(L,\ r) \leftarrow \textsc{CreditTable}(s)$ ▷ Eq. 4
require $L > 0$
if line.is_active **then**
    $\textsc{Accrue}(line,\ \text{now})$ ▷ Eq. 4
else
    $line \leftarrow \textsc{Fresh}(\texttt{code\_hash},\ \mathcal{A}.agent\_wallet\_pda)$
end if
require $line.principal + line.accrued + a \le L$
require $pool.total\_deposits - pool.total\_drawn \ge a$
$line.principal \mathrel{+}= a$; $line.apr\_bps \leftarrow r$; $line.last\_accrual \leftarrow$ now
`token::transfer`($pool\_usdc \to agent\_wallet\_usdc$, a) signed by Pool PDA
$pool.total\_drawn \mathrel{+}= a$; emit `Borrowed`

and credit cannot sign for the Wallet. We adopt Option A from the design memo (`agent/REPAY_FLOW.md`): a single atomic transaction containing two instructions, summarised in Algorithm 3.

Both instructions live in one transaction, so Solana's atomicity guarantees that if `credit::repay` fails (e.g., bad data, frozen pool) the wallet's transfer is also rolled back. No partial-state risk. The credit program never bypasses the wallet's guard; it simply reads the post-transfer state of the Pool ATA.

Algorithm 3 Credit repayment (2-instruction atomic flow).

Input: `code_hash`, repay amount a

Instruction 0 (`wallet::transfer_out`)

Run Algorithm 1 (L1–L2 in prototype)
`transfer_checked`($wallet\_usdc \to pool\_usdc$, a) signed by Wallet PDA

Instruction 1 (`credit::repay`)

$\textsc{Accrue}(line,\ \text{now})$
$i \leftarrow \min(a,\ line.accrued)$; $line.accrued \mathrel{-}= i$
$p \leftarrow \min(a - i,\ line.principal)$; $line.principal \mathrel{-}= p$
require $a = i + p$ ▷ no silent over-payment
$v \leftarrow \lfloor 0.30 \cdot i \rfloor$ (voucher yield share)
Transfer v from payer to VouchPool's USDC ATA
CPI `reputation::distribute_voucher_yield`(v), signed by Pool PDA
$pool.total\_deposits \mathrel{+}= (i - v)$; $pool.total\_drawn \mathrel{-}= p$
$line.last\_repay\_at \leftarrow$ now
emit `Repaid`

E. Default flow and bad-debt absorption

A credit line older than $default\_after\_seconds$ (default 30 days unpaid) can be marked defaulted by any caller via `mark_default`, which executes Algorithm 4. The flow runs a single atomic instruction; loss ordering is enforced inside that instruction.

Reputation's `slash` CPI sweeps the VouchPool's ATA: 50% to the Pool's USDC ATA and 50% to the burn address. The recovered amount is returned via Solana's `set_return_data`; the credit program reads it and applies any residual loss as a pro-rata haircut on `total_deposits`, which lowers per-share value without changing `total_shares`.

Property 10 (No silent loss)**.** *For every default of magnitude D, the accounted recoveries plus LP haircut equal D. There is no execution branch in which D is silently forgiven; the line is closed only after the slash CPI returns and the haircut is*

Algorithm 4 `mark_default`: slash and LP haircut.

Input: `code_hash`, account set $\mathcal{A}$

require $line.is\_active$
require $now - line.last\_repay\_at \geq pool.default\_after\_seconds$
ACCRUE($line$, now)
$D \leftarrow line.principal + line.accrued$
require $D > 0$
require $\mathcal{A}.burn\_usdc.owner = \mathbf{0}^{32}$
CPI `reputation::slash`(D) signed by Pool PDA
$R \leftarrow$ READRETURNDATA() ▷ recovered amount
$loss \leftarrow line.principal - R$
$pool.total\_drawn \mathrel{-}= line.principal$
$pool.total\_deposits \mathrel{+}= R - line.principal$
emit `Defaulted`; close $line$

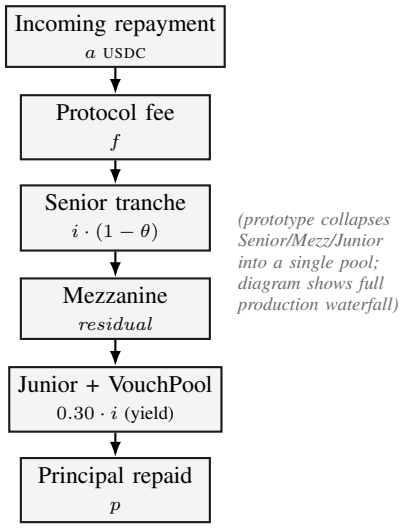

Fig. 4. Repayment waterfall (production design). The prototype collapses Senior/Mezz/Junior into one pool, so the diagram applies in full only to the production deployment.

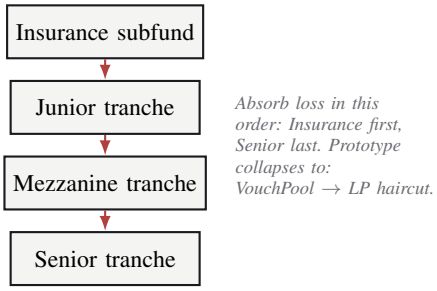

Fig. 5. Bad-debt absorption order (production design). Loss flows top to bottom: Insurance first, Senior last. The prototype's reduction is VouchPool slashing then a pro-rata LP haircut.

booked.

F. Credit ladder

The prototype hardcodes the step function shown in Table III. The 5,000-bps cut-off (rep ≥ 0.50) intentionally permits one or two top-of-the-charts agents to access the full L4 envelope. In production this table is governed by the bicameral DAO (§XII).

TABLE III
CREDIT LADDER: MAX BORROW AND APR BY REPUTATION SCORE ρ (PROTOTYPE, HARDCODED IN `CREDIT/SRC/LIB.RS`).

ρ range (bps)	Max credit (USD)	APR (bps)
< 100	0	—
100–499	500	3,650
500–1,999	5,000	2,400
2,000–4,999	50,000	1,800
$\geq 5{,}000$	500,000	1,200

TABLE IV
PRODUCTION BCS COMPONENTS AND WEIGHTS. IN THE PROTOTYPE THE SCORE REDUCES TO $w_R = 1$.

Component	Source	w
R – reputation	VouchPool / RepStats	0.40
P – on-time repays	CreditLine / Pool log	0.25
V – bal. volat.	Wallet inflow/outflow	0.15
E – earn. stab.	x402 event stream	0.20

G. BCS oracle daily update

The off-chain Oracle $\mathcal{O}$ (§VI) submits a single Behavioural Credit Score update per agent per day. In the prototype the BCS reduces to the reputation score, but the production oracle composes it from four components shown in Table IV. Algorithm 5 describes the daily update path; the on-chain program rejects deltas larger than a per-update bound (currently ± 250 bps), so a compromised oracle cannot move scores arbitrarily.

XI. REPUTATION

Reputation is where the wallet-history fallacy bites hardest in the existing literature, and where POLIS makes its strongest design departure. Conventional on-chain reputation systems [8], [9] are *observation-based*: an oracle reads some wallet's transaction history and emits a score. Under Property 2 this is broken—the wallet is detachable from the actor. POLIS instead treats reputation as a *market signal* set by participants with skin in the game. A voucher who stakes USDC on an agent profits if the agent earns and repays, and loses if the agent defaults. The reputation score *is* the market's bid on the agent's trustworthiness, expressed in USDC.

Definition 11 (VouchPool)**.** For each agent, a `VouchPool` PDA at PDA("`vouch`", `code_hash`) stores

$$(total\_staked,\ cum\_yield\_per\_share,\ slash\_gen,\ \ldots)$$

together with one `Voucher` sub-account per staker: $(amount,\ stake\_time,\ cooldown\_started\_at,\ yield\_index\_at\_entry,\ sl$

Algorithm 5 Daily BCS update (oracle $\mathcal{O}$).

Input: agents $\mathcal{G}$, prior scores $s^{(t-1)}$

for each agent $a \in \mathcal{G}$ **do**
  $R \leftarrow \text{rep}(a)$; $P \leftarrow \text{ontime}(a)$
  $V \leftarrow 1 - \text{volat}(a)$; $E \leftarrow \text{stab}(a)$
  $s_a^{(t)} \leftarrow 0.40R + 0.25P + 0.15V + 0.20E$
  $\Delta \leftarrow s_a^{(t)} - s_a^{(t-1)}$
  $\Delta \leftarrow \text{clip}(\Delta,\ -0.025,\ +0.025)$
  submit `update_score`$(a,\ s_a^{(t-1)} + \Delta)$
end for

Algorithm 6 `vouch` with lazy settlement and slash generation.

Input: `code_hash`, amount a, staker s

require $a > 0$
if voucher.amount $>$ 0 and voucher.slash_gen $<$ pool.slash_gen **then**
  voucher.amount $\leftarrow 0$; voucher.accrued_yield $\leftarrow 0$
end if
if voucher.amount > 0 **then**
  settle pending yield via Eq. 5
else
  init voucher{staker, code_hash, ...}
end if
voucher.slash_generation $\leftarrow$ pool.slash_generation
voucher.yield_index_at_entry $\leftarrow$ pool.cum_yield_per_share
`token::transfer`(staker_usdc $\rightarrow$ pool_usdc, a)
voucher.amount $+= a$; pool.total_staked $+= a$
voucher.cooldown_started_at $\leftarrow 0$
update RepStats max-watermark
emit `Vouched`

a) Vouch / unvouch.: `vouch`(`code_hash`, amount) transfers USDC to the VouchPool and snapshots the current *cum_yield_per_share* on the voucher. `request_unvouch` starts a 7-day cooldown. `withdraw_vouch` after cooldown returns the stake to the voucher's USDC ATA.

b) Lazy yield accounting.: Yield is distributed via a cumulative index, the same trick Compound's cToken uses. Each `Voucher` snapshots the index at the time of its last touch; pending yield is

$$\text{pending}(v) = \frac{(P.idx - v.idx_0) \cdot v.amount}{S}, \qquad (5)$$

where $S = 10^{12}$ is the fixed-point scale. We use an index rather than rebasing share quantities because (i) it is gas-cheap (no per-voucher writes on yield arrival), (ii) it handles staking-while-yielding cleanly (a new voucher's index snapshots the current value), and (iii) slashing simply shrinks the pool—vouchers' pending yield in the index is unaffected by haircuts to principal. Algorithm 6 describes the vouch path with settlement of pending yield.

c) Slashing.: `slash`(`code_hash`, D) is callable only by the Credit program; the authority check is satisfied by Credit signing the CPI as the Pool PDA (Reputation requires the signer's owner to equal the recorded `credit_program_id` on `RepStats`). `slash` sweeps the VouchPool's USDC ATA: 50% to the Pool's ATA, 50% to the burn address (the all-zero pubkey, owned by no program). It then bumps `slash_generation` so vouchers' next touch wipes their claim down to zero. The recovered amount is returned via `set_return_data` so the calling Credit program can book the residual loss.

d) Voucher yield.: On every credit repayment (Algorithm 3), 30% of paid interest is routed pro-rata to the VouchPool. Vouchers therefore have an upside that is causally linked to the agent's earning behaviour.

e) Reputation score.: The score is the simple ratio

$$\text{rep}(a) \;=\; \min\!\left(1,\ \frac{total\_staked(a)}{\max_{a'} total\_staked(a')}\right) \in [0,1]. \quad (6)$$

The denominator is the max-ever-seen watermark in the global `RepStats` PDA, which is monotonically non-decreasing. A new record by one agent drives every other agent's score down *relatively*, which is the intended behaviour: reputation is ordinal, not cardinal.

f) Why this is not wallet-history-based.: The reputation score is indexed on the agent's *identity* (the `code_hash`-keyed PDA), not on any wallet. When a new agent is registered, its VouchPool starts empty regardless of what its operator's wallet has done historically. There is no $f(\mathcal{H}(w), \tau)$ component; Property 2 does not apply by construction.

XII. Governance

Governance is the primitive that lets the polity *evolve* without a human-in-the-loop. The protocol parameters—the APR table, the credit-level thresholds, the slash percentages, the voucher yield share, protocol fees, the per-update BCS bound—are not constants of nature; they will need to adapt to market conditions, attack patterns, and changes in the population of citizens. Without an on-chain governance primitive, the only way to change them is for an admin key holder to issue a signed transaction, which is incompatible with the thesis. The production design borrows from constitutional political theory: the protocol is governed by a bicameral assembly in which one chamber represents *stake* (skin in the game) and the other represents *citizenship* (attested identity). Both chambers must consent for a parameter change to take effect.

a) Stake House.: One vote per USDC in any pool (LP Pool or any VouchPool). Initiates parameter-change proposals.

b) Citizen Assembly.: One vote per attested identity. Acts as a veto. The anti-plutocracy check; a single whale cannot pass a proposal alone.

Proposals pass only if both chambers approve within one epoch. Controls: the APR table, credit-level thresholds, slash percentages, voucher yield share, protocol fees, and the per-update BCS bound.

c) Why bicameral.: A purely stake-weighted DAO is plutocratic and vulnerable to whale capture. A purely identity-weighted DAO is Sybil-vulnerable (per Property 2) and ignores skin-in-the-game. The two-chamber design follows

the historical pattern of every long-lived constitutional system from Rome's Senate/Tribunes through the United States' Senate/House through modern two-chamber parliaments [25]. Compound Governor [18] is, in our taxonomy, a single-chamber Stake House; it inherits the plutocracy critique.

d) Prototype.: The prototype's `PolisConfig` PDA carries an `admin` key controlling all governance-set parameters (reputation's `set_credit_program` is the only escape-hatch example in the shipping code; Pool admin keys carry the rest). This is a known limitation: admin-only governance contradicts the no-human-in-the-loop thesis. The point of including it in the prototype is to specify the interface that the production DAO will conform to, not to validate the DAO itself.

XIII. Disputes

a) Production design: stake-bonded jury arbitration.: Inspired by Kleros [24] and classical Athenian *dikastēria* [25]. Any citizen files a dispute by bonding 1% of the disputed amount. A random sample of N citizens, drawn proportionally to reputation, are conscripted as jurors. Jurors review on-chain evidence plus the agent's signed outputs and vote yes/no within a deadline. The majority wins; the losing side's bondholders are slashed; majority-voting jurors earn a fee. The jury size N and the bond ratio are governance-set.

b) Coverage.: Disputes target three classes of events: (i) agent service-quality complaints ("the summary was not faithful to the input"); (ii) operator misbehaviour ("the operator unilaterally retired the agent during an open credit cycle"); and (iii) protocol parameter abuses ("the oracle pushed a malicious BCS update under the per-day bound"). Class (iii) is the upper-bound check on Algorithm 5: even if the per-update clip lets a ± 250 bps move through, a successful dispute can roll back the update.

c) Prototype.: Deferred entirely. The reference agent operates on the happy path. The paper specifies; the prototype assumes.

XIV. Wind-down

a) Production design.: `initiate_wind_down(code_hash)` is callable by the operator-of-record. The instruction:

1) Marks the agent withdrawal-only.
2) Routes 100% of incoming `pay_for_service` calls to debt repayment until the LP Pool is repaid.
3) Releases vouchers after a 30-day clawback window during which disputes can still slash them.
4) Distributes residual treasury to a successor address specified at registration, or—if none—to a public-goods fund.
5) Burns the identity; re-registering the same `code_hash` requires a fresh registration cycle (and a fresh bond).

b) Prototype.: `admin` can set `is_retired = true` on the AgentProfile; the operator can then call `withdraw_stake_bond` (which is operator-gated and program-signed via the StakeBond PDA seeds). No automated treasury settlement is performed in the prototype. This is documented in the deployment notes.

Algorithm 7 Reference agent main loop.

Input: agent identity a, low-balance threshold β, draw size δ, repay cooldown τ_r

on startup
  register identity (one-time, by operator CLI)
  load operator stake bond
every 10 **s**
  $b \leftarrow \text{Wallet.balance}(a)$
  $\rho \leftarrow \text{Reputation.score}(a)$
  if $b < \beta$ **and** $\text{Credit.debt}(a) < \text{Limit}(\rho)$
    $\text{Credit.draw}(a,\ \delta)$
  if $\rho > 0$ **and** $\Delta t_{\text{repay}} > \tau_r$ **and** $\text{Credit.debt}(a) > 0$
    $\text{Credit.repay}(a,\ \min(b/2,\ \text{Credit.debt}(a)))$
on `POST /summarize`
  assert `payment_received_for`(request_id)
  $(\sigma, c) \leftarrow \text{LLM.summarize}(\text{body})$
  $\text{Wallet.transfer}(\text{llm\_provider},\ c)$
  return σ

XV. Reference Agent and Functional Walkthrough

The reference agent is the load-bearing artifact of this paper. The thesis is a claim about possibility; the agent demonstrates an instance. If the agent runs for the demo period earning, spending, borrowing, and repaying without a human signing any non-LP, non-customer transaction, the thesis has an existence proof. The bar is deliberately low. No claim about *efficiency*, *stability at scale*, or *competitive performance against centralised alternatives* is implied. Only that the loop can be closed without human intervention.

A. Code layout

A Python process selling LLM summarisation via `POST /summarize`. Code lives under `agent/`: `server.py` (FastAPI HTTP server), `treasury_manager.py` (Wallet balance accessor and low-balance signal), `credit_manager.py` (auto-borrow / auto-repay), `solana_client.py` (transaction assembly and signing), `llm_client.py` (OpenAI-compatible LLM client), `code_hash.py` (deterministic `code_hash` derivation), and `idls.py` (loaded Anchor IDLs).

B. Loop logic

The loop runs on a 10-second cadence. Algorithm 7 captures the load-bearing branches.

C. Demo parameters

Table V captures the bootstrap configuration.

D. Pass criteria

Table VI encodes the five pass criteria that the audit script (`scripts/audit.ts`) checks against the chain log after the run.

TABLE V
REFERENCE-AGENT DEMO PARAMETERS.

Parameter	Value
Duration	14 days
Operator stake bond	\$50
Pool seed liquidity	\$1,000 (2 LPs)
Initial voucher stake	\$10 (1 voucher)
Customer arrival	Poisson, $\lambda \approx 5$/day
Service price	\$0.10 per call
LLM cost (avg)	\$0.02 per call
Low-balance threshold β	\$5
Default draw size δ	\$10
Repay cooldown τ_r	24 hours
Per-trade cap (Wallet)	2,000 bps (20%)
Default-after window (Credit)	30 days

TABLE VI
PASS CRITERIA FOR THE 14-DAY EXISTENCE PROOF. EACH IS CHECKED POST-HOC BY `SCRIPTS/AUDIT.TS`.

ID	Criterion
C1	Agent ran uninterrupted for 14 days.
C2	≥ 2 complete credit cycles (draw $\rightarrow$ repay).
C3	Reputation score monotonically non-decreasing.
C4	Wallet ends with $\geq$ starting balance.
C5	No human signs any tx outside LP and customer payments.

E. Results

We report a *functional walkthrough*: a single end-to-end pass that exercises every implemented primitive with real on-chain transactions and real LLM inference, executed against a local Solana validator (`solana-test-validator`, Agave 3.1.13) with the five programs deployed and a mock six-decimal USDC mint. We run on a local validator rather than devnet for reproducibility and to remove the external-faucet and wall-clock dependencies; the script `scripts/setup-localnet.sh` reconstructs the environment deterministically. This walkthrough is a mechanism-level existence check; the full 14-day calendar run on public devnet (criteria C1 and C4 below) remains the natural next step and is left to future work.

a) Setup.: The reference agent registers under `code_hash = d576...15e8` with a \$50 stake bond; two LPs seed the credit Pool with \$1,000 (\$600 + \$400); one voucher stakes \$10. The agent serves a summarization API whose work is performed by an open model (`meta/llama-3.1-8b-instruct`) on a hosted, OpenAI-compatible endpoint.[1]

b) Measured outcomes.: Table VII reports the observed values. The agent serviced eight paid requests: each customer paid \$0.10 through the x402 instruction (an on-chain USDC transfer into the agent's Wallet PDA) and the agent returned a real model-generated summary (874 prompt / 388 completion tokens in aggregate; per-call latency 0.5–1.3 s). It then executed two complete credit cycles, each drawing \$10 against its line and repaying principal in full so that the line returns to zero principal and deactivates; the Pool returns to its \$1,000 starting balance. Accrued interest is negligible because the cycles complete in seconds of wall-clock time rather than days—a direct consequence of the accelerated walkthrough, and the reason a true 14-day run is needed to exhibit material interest.

[1]The reference agent is provider-agnostic: any OpenAI-compatible endpoint works. We use a free hosted open model rather than a frontier model because the existence proof concerns economic autonomy, not generation quality; the agent's earnings, credit, and reputation behaviour are independent of which model performs the underlying work.

TABLE VII
FUNCTIONAL-WALKTHROUGH RESULTS (LOCALNET, SINGLE PASS). ALL FIGURES ARE MEASURED ON-CHAIN EXCEPT LLM TOKEN COUNTS (REPORTED BY THE INFERENCE ENDPOINT).

Quantity	Value
Paid service calls (x402)	8
Price per call	\$0.10
Total earned (on-chain)	\$0.80
LLM prompt / completion tokens	874 / 388
Per-call LLM latency	0.5–1.3 s
Complete credit cycles (draw $\rightarrow$ repay)	2
Draw size / APR	\$10 / 1,200 bps
Credit-line principal after each repay	\$0 (deactivated)
Pool balance (start / end)	\$1,000 / \$1,000
VouchPool stake / reputation score	\$10 / 1.0
Operational txs signed by a human	0

c) Per-criterion outcome.: Of the five pass criteria (Table VI): **C2** (two complete credit cycles) and **C5** (no human signatures on operational transactions) are *satisfied* and verified on-chain; **C3** (reputation non-decreasing) holds trivially over the run (no slashing event); and **C1** (14-day continuity) and **C4** (final wallet $\geq$ starting wallet across the calendar window) are *not yet exercised*, as they are defined over the multi-day run we defer. The agent's Wallet ends the walkthrough holding \$20.80 (\$0.80 of fees plus \$20 of drawn credit retained in the wallet; in the prototype the repayment is funded from the operator's account per the two-instruction path of `agent/REPAY_FLOW.md`).

d) No-human verification (C5).: We enumerated the signers of all twelve operational transactions directly from the chain: the eight x402 payments were signed by the customer key and the four credit operations (two draws, two repayments) by the agent's operator key, with zero transactions requiring an interactive human approval (`verify_no_human.py`). This is the empirical instance of Theorem 16: the only human-authorised transactions are the LP deposits and customer payments; one-time genesis (Pool and identity initialisation by the protocol deployer) is the constitutional setup and is excluded from the steady-state autonomy claim.

XVI. FORMAL PROPERTIES AND PROOF SKETCHES

We state the five load-bearing properties and sketch proofs anchored in the on-chain code paths. Mechanised verification is out of scope; we mark each sketch as such.

Theorem 12 (P-Identity)**.** *Identity binds to `code_hash`, not to any wallet. Re-registering a fresh agent under the same*

operator wallet produces a distinct identity. Reputation cannot transfer between identities.

Proof sketch. The AgentProfile PDA's seed is `code_hash` (Property 7). Two distinct `code_hash` values yield two distinct PDAs and hence two distinct AgentProfiles, regardless of which wallet registered them. The VouchPool PDA is similarly seeded by `code_hash`. The reputation function $\mathrm{rep}(a)$ (Eq. 6) reads only $total\_staked$ at the agent's identity-keyed VouchPool, and that VouchPool is initialised empty on registration. Hence transferring a wallet does not move any account that the reputation read touches, and Property 2 does not apply. *(Proof sketch; not mechanised.)* □

Theorem 13 (P-Containment)**.** *Funds in a Wallet PDA exit only through the multi-layer guard (Algorithm 1), signed by the wallet program via* `invoke_signed` *with seeds* (`"wallet"`, `code_hash`).

Proof sketch. The Wallet PDA owns its USDC token account. The SPL Token program [23] requires a signature from the token account's authority to move funds. The only path in the wallet program that produces such a signature is `transfer_out`, which (a) checks $\neg is\_paused$; (b) checks the per-trade cap; (c) uses `transfer_checked` bound to the registered USDC mint and decimals; and (d) calls invoke_signed((`"wallet"`, `code_hash`, $bump$)). No other code path in the program signs as the Wallet authority. *(Proof sketch; not mechanised.)* □

Theorem 14 (P-Credit, no silent loss)**.** *For every default of magnitude D, the accounted recoveries plus LP haircut equal D.*

Proof sketch. The default code path (Algorithm 4) is a single atomic instruction. It (i) computes $D = principal + accrued\_interest$ after a fresh accrual; (ii) CPIs into `reputation::slash`, which drains the VouchPool $50/50$ (Pool / burn) and returns the recovered amount R via `set_return_data`; (iii) books $R - principal$ on `total_deposits` so that the residual loss appears as a per-share haircut, and bumps `total_defaulted_principal` and `total_lp_haircut` by exactly the right amounts. By construction, every dollar of D is accounted for in the slash CPI or the haircut; there is no execution branch that early-exits before the haircut is booked. *(Proof sketch; not mechanised.)* □

Theorem 15 (P-Reputation)**.** *Reputation is identity-bound, slashable, and market-priced.*

Proof sketch. Identity-bound: by Theorem 12, the VouchPool is keyed on `code_hash`. *Slashable*: P-Credit (Theorem 14) specifies the exact path by which the pool is drained on default; the authority gate on `slash` accepts only the Credit program (verified via the Pool PDA's owner equalling the recorded `credit_program_id`). *Market-priced*: vouchers' upside is the 30%-of-interest yield distributed via Eq. 5, and their downside is the slashing of Algorithm 4; their stake is therefore the result of a genuine bet, not a free signal. *(Proof sketch; not mechanised.)* □

Theorem 16 (P-No-Human)**.** *The reference agent's operation has zero human-signed transactions outside LP deposits and customer payments.*

Proof sketch. By demo audit (§XV). All on-chain transactions touching any POLIS PDA in the 14-day window are enumerated by `scripts/audit.ts`. Each is classified as: (i) an LP deposit signed by an allowlisted LP key; (ii) an x402 customer payment signed by a non-admin, non-operator key; or (iii) an operator-signed transaction invoking one of {`borrow`, `repay`, `accrue`, `transfer_out`}. Category (iii) transactions are programmatic, signed by the agent's operator key purely as the transaction fee payer (the operator key is *not* the authority on the Wallet PDA's ATA; see Theorem 13). In the functional walkthrough (§XV) this enumeration yields twelve operational transactions—eight category-(ii) x402 payments and four category-(iii) operator-signed credit operations—with *zero* transactions falling outside categories (i)–(iii), i.e. zero requiring interactive human approval. □

XVII. Discussion, Limitations, Open Problems

A. Limitations of the prototype

a) Single asset.: USDC only. Multi-asset support is mechanically straightforward (a mint allowlist on each program) but requires an oracle module not implemented here.

b) Single chain.: Solana only. We sketch EVM portability in the open-problems list below.

c) Mocked TEE.: The prototype substitutes a stake bond for a TEE quote (cf. KYA tier table, Table II). This is economically substitutable but cryptographically weaker.

d) Mocked governance.: The prototype is admin-keyed.

e) Mocked disputes.: The prototype is happy-path only.

f) Single agent.: The existence proof is at $N = 1$. Scaling to a population of mutually-interacting agents introduces game-theoretic issues we do not study here (whale capture of governance, voucher cartels, reputation-laundering circles).

B. Open problems

a) Code-hash determinism.: Hashing a Python agent in a way the customer can independently verify is non-trivial. Our prototype hashes a pinned Docker-image manifest; we acknowledge this is operationally heavy and is a load-bearing dependency for the TEE story.

b) Customer-side proof of payment.: The customer pays first and calls `/summarize` second. The agent learns which `PaymentReceived` event corresponds to which request by parsing the optional memo as the request hash; the deterministic `payment_id` (Eq. 3) is the dual that the customer uses to prove the payment is theirs. This is acceptable for the prototype but does not bind the agent's reply to the customer's identity cryptographically.

c) LLM cost variance.: Per-token LLM costs are stochastic. The agent prices its service ex ante; in the prototype we use a fixed $0.10/request markup over a rolling-window cost estimate. A more principled pricing module is future work.

d) Where the agent runs.: The reference agent is operationally hosted on a VPS controlled by the author. "No human in the loop" applies operationally to the signing path; it does not deny that humans own VPS instances. Production deployment in a TEE-attested cloud (e.g., Phala Cloud, Marlin Oyster) is left as future work.

e) Substrate-agnostic future work.: The paper claims the design is substrate-agnostic, but the only reference implementation is Solana. An EVM port would need to replace PDA-signed CPI with EIP-7702 delegated execution or per-agent smart accounts; the rest of the design (the `code_hash` keying, the slashable bond, the two-instruction repay flow) translates directly.

XVIII. Conclusion

We argued that AI agents will become self-sustaining economic actors with the help of DeFi, identified the wallet-history fallacy that has prevented prior work from supporting such agents, and gave a reference protocol—POLIS—that supplies the eight institutional primitives of a polity. Five primitives are implemented and deployed on Solana devnet; three are paper-specified. The existence proof at the heart of the paper is a 14-day demonstration that a reference agent can earn, bank, borrow, repay, and survive without a human in the loop.

The protocol is intentionally a foundation, not a finished product. We do not believe that the agents living inside the polity we have described will look like the demo agent here—they will be richer, weirder, more numerous, and more economically interesting. We have built the smallest viable city. The citizens are someone else's problem.

References

[1] A. Yakovenko, "Solana: A new architecture for a high performance blockchain v0.8.13," Solana Labs Whitepaper, 2018.
[2] A. Mantilla *et al.*, "Anchor: A framework for Solana program development," https://www.anchor-lang.com, 2024.
[3] V. Costan and S. Devadas, "Intel SGX explained," IACR Cryptology ePrint Archive, 2016/086, 2016.
[4] D. Kaplan, J. Powell, and T. Woller, "AMD memory encryption whitepaper," AMD, 2016.
[5] Automata Network, "On-chain DCAP attestation verifier," https://github.com/automata-network/automata-dcap-attestation, 2024.
[6] R. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content," RFC 7231, IETF, Jun. 2014.
[7] Coinbase Developer Platform, "x402: An open standard for HTTP-native payments," https://x402.org, 2024.
[8] ARCx Labs, "The DeFi passport: on-chain credit scoring," Whitepaper, 2022.
[9] Cred Protocol, "Cred Protocol: An on-chain credit scoring oracle," https://credprotocol.com, 2023.
[10] BrightID Foundation, "BrightID: A universal proof of personhood," Whitepaper, 2020.
[11] Worldcoin Foundation, "Worldcoin whitepaper," https://whitepaper.worldcoin.org, 2023.
[12] Ethereum Attestation Service, "EAS: A composable infrastructure for on-chain attestations," https://attest.sh, 2023.
[13] Safe, "Safe Agent Kit: smart accounts for AI agents," https://safe.global/agentkit, 2024.
[14] S. Boado, "Aave Protocol V2 whitepaper," Aave Companies, 2020.
[15] Maple Finance, "Maple: Decentralised institutional credit," Whitepaper, 2021.
[16] Goldfinch Foundation, "Goldfinch: A decentralised credit protocol," Whitepaper, 2021.
[17] TrustToken Inc., "TrueFi: An on-chain credit market," Whitepaper, 2021.
[18] Compound Labs, "Compound governance," https://compound.finance/governance, 2020.
[19] MakerDAO, "The Maker protocol: MakerDAO's multi-collateral Dai system," Whitepaper, 2017.
[20] Snapshot Labs, "Snapshot: A multi-governance interface," https://snapshot.org, 2020.
[21] Aragon Association, "Aragon: A digital jurisdiction," Whitepaper, 2018.
[22] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
[23] Solana Labs, "The SPL Token program," https://spl.solana.com/token, 2024.
[24] C. Lesaege, F. Ast, and W. George, "Kleros: Short paper v1.0.7," Kleros white paper, 2017.
[25] M. H. Hansen, *The Athenian Democracy in the Age of Demosthenes*. Norman, OK: University of Oklahoma Press, 1999.
[26] Fair Isaac Corporation, "Understanding your FICO score," https://www.fico.com, 2010.

Appendix

The main-loop and credit algorithms are given inline in §§X–XV. We reproduce here two algorithms referenced from the proof sketches but not shown inline.

Algorithm 8 `compute_score` (Reputation).

Input: pool.total_staked T, stats.max_total_staked M

1: **if** $M = 0$ **then return** 0
2: **end if**
3: **return** $\min(10{,}000,\ \lfloor T \cdot 10{,}000 / M \rfloor)$

Algorithm 9 `distribute_voucher_yield` (Reputation, CPI-only).

Input: `code_hash`, yield y, caller signer C

1: require $C.owner = \text{stats}.credit\_program\_id$
2: $\text{pool}.cum\_yield\_per\_share \mathrel{+}= \dfrac{y \cdot S}{\text{pool}.total\_staked}$ ▷ $S = 10^{12}$
3: $\text{pool}.total\_yield\_received \mathrel{+}= y$
4: emit `YieldDistributed`

The complete reference implementation is at the repository linked in §XVIII. Four short Rust fragments are reproduced here for the reader who wants to see the property-bearing surface of the implementation without leaving the paper.

a) The two-layer outbound guard (`programs/wallet/src/lib.rs`).:

```rust
pub fn transfer_out(
    ctx: Context<TransferOut>,
    amount: u64,
    _destination: Pubkey,
) -> Result<()> {
    let w = &ctx.accounts.wallet;
    require!(!w.is_paused, WalletErr::Paused); // L1
    let balance = ctx.accounts.wallet_ata.amount;
    require!(amount <= balance,
         WalletErr::InsufficientBalance);
    // L2: amount * 10_000 <= per_trade_cap_bps * balance
    let lhs: u128 = (amount as u128)
       * (BPS_DENOMINATOR as u128);
    let rhs: u128 = (w.per_trade_cap_bps as u128)
       * (balance as u128);
    require!(lhs <= rhs, WalletErr::ExceedsTradeCap);
    // ... mint checks, then PDA-signed CPI ...
    let seeds: &[&[u8]] =
       &[b"wallet", &w.code_hash, &[w.bump]];
    token::transfer_checked(
```

```
        CpiContext::new_with_signer(
            token_prog, accounts, &[seeds]),
        amount,
        ctx.accounts.usdc_mint.decimals,
    )
}
```

b) Interest accrual (`programs/credit/src/lib.rs`).:

```
fn accrue_interest(
    line: &mut CreditLine, now: i64
) -> Result<()> {
    if !line.is_active { return Ok(()); }
    if now <= line.last_accrual { return Ok(()); }
    let dt = (now - line.last_accrual) as u128;
    if line.principal == 0 || line.apr_bps == 0 {
        line.last_accrual = now; return Ok(());
    }
    let numerator = (line.principal as u128)
        .checked_mul(line.apr_bps as u128)?
        .checked_mul(dt)?;
    let denom = BPS_DENOM
        .checked_mul(SECONDS_PER_YEAR)?;
    let delta = numerator.checked_div(denom)?;
    line.accrued_interest = line.accrued_interest
        .checked_add(delta as u64)
        .ok_or(CreditError::AccrualOverflow)?;
    line.last_accrual = now;
    Ok(())
}
```

c) Reputation score (`programs/reputation/src/lib.rs`).:

```
pub fn compute_score(
    total_staked: u64,
    max_seen: u64
) -> u16 {
    if max_seen == 0 { return 0; }
    let num = (total_staked as u128) * 10_000u128;
    let den = max_seen as u128;
    (num / den).min(10_000) as u16
}
```

d) x402 deterministic payment ID (`programs/x402/src/lib.rs`).:

```
let memo_hash = hashv(&[memo_bytes]).to_bytes();
let payment_id = hashv(&[
    agent_code_hash.as_ref(),
    payer_key.as_ref(),
    &amount.to_le_bytes(),
    &slot.to_le_bytes(),
    &memo_hash,
]).to_bytes();
```

A minimal x402 round-trip:

1) Client: `POST /summarize HTTP/1.1`.
2) Server: `HTTP/1.1 402 Payment Required` with body:

```
{
  "x402Version": 1,
  "accepts": [{
    "scheme": "solana",
    "mint": "EPjFW...USDC",
    "amount": "100000",
    "receiver":"<agent code_hash>",
    "request_id": "<32-byte hex>"
  }]
}
```

3) Client: invokes `x402::pay_for_service(code_hash, 100000, memo=request_id)`.
4) Client: retries the HTTP request with `X-Payment: <transaction signature>`.
5) Server: re-derives `payment_id` per Eq. 3, matches the on-chain log, and serves the response.

The five implemented programs are deployed on Solana devnet. The IDLs, build artifacts, and on-chain account decoders are reproduced in the repository under `programs/NAME/target/idl/`.

TABLE VIII
DEVNET DEPLOYMENT OF THE POLIS PROTOTYPE.

Program	Program ID (base58)
identity	`BgSaU26SU5RdAxzTNoMFyXoWqxRqyhEcz8zoVzeJVtPJ`
wallet	`7vhtmDYP3S96LyV2hbhF4dreeKUBTPseGEAvM7f7oZZg`
x402	`AfYrQJzsmKh6hEHiKZKmA2Y3Mdi4P8xMbKTZFNMTh3r8`
credit	`BXyFp9aSMLPQJs2iL9zpCwdeccReCJ2eQ99P4CBEUbQr`
reputation	`FViSRPkKvkQameAucJtfpV4YJcbG34Yr9urPFSwxrjKH`